ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор		Ю.А. Скобцов
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Решение задач комбинаторной оптимизации с помощью генетических алгоритмов на примере задачи укладки рюкзака

по курсу: ЭВОЛЮЦИОННЫЕ МЕТОДЫ ПРОЕКТИРОВАНИЯ ПРОГРАММНО-ИНФОРМАЦИОННЫХ СИСТЕМ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Индивидуальное задание по варианту:

Задание

1. Реализовать с использованием генетических алгоритмов решение задачи укладки рюкзака простой сложности по индивидуальному заданию согласно номеру варианта (см. таблицу 1) использованием соответствующего кодирования.

2. Сравнить найденное решение с представленным в условии задачи оптимальным решением.

3. Реализовать с использованием генетических алгоритмов решение задачи укладки рюкзака повышенной сложности по индивидуальному заданию согласно номеру варианта соответствующего кодирования (см. таблицу 2).

4. Представить графики сходимости решения с различными параметрами генетического алгоритма: мощности популяции, значений вероятностей различных видов скрещивания, мутации.

5. Проанализировать время выполнения и точность нахождения результата в зависимости от значений вероятностей различных видов скрещивания, мутации.

Вариант	Номер тестовых данных задания1 простой сложности	Номер тестовых данных задания 2 повышенной сложности	Вид кодирования решения	Контрольный вопрос
19	5	2	двоичное	9

P05 is a set of 8 weights and profits for a knapsack of capacity 104.

Набор 5
Емкость
104

Веса
25
35
45
5
25
3
2
2
Стоимости
350

400
450
20
70
8
5
5

Оптимум
1
0
1
1
1
0
1
1

Набор 2 некоррелированные

50
13810

957 491
948 925
57 501
85 832
836 354
736 883
670 900
309 462
606 568
607 921
582 724
159 487
431 222
394 325
724 700
995 167
950 908
545 269
445 912
269 310
36 958
28 707
465 505
319 518
381 652
892 588
526 312
561 208
237 512
24 935

326 624
137 76
511 821
999 726
675 908
182 192
894 745
797 59
735 653
907 274
763 227
790 876
354 107
981 523
962 854
162 245
755 211
716 881
462 423
531 717

Краткие теоретические сведения:

Генетические алгоритмы (ГА) представляют собой эвристический метод оптимизации, вдохновленный процессами эволюции в природе. Они работают с популяцией возможных решений, которые представляются в виде хромосом, где каждая хромосома кодирует набор параметров задачи. Через процессы селекции, скрещивания и мутации алгоритм постепенно улучшает популяцию, отбирая лучшие решения на основе функции приспособленности, которая оценивает качество каждого решения. В задаче укладки рюкзака ГА используются для выбора оптимального набора предметов с учетом ограничений по весу, стремясь максимизировать общую стоимость. Этот подход особенно полезен для сложных комбинаторных задач, где традиционные методы могут быть неэффективны.

Программа и результаты выполнения индивидуального задания с комментариями и выводами:

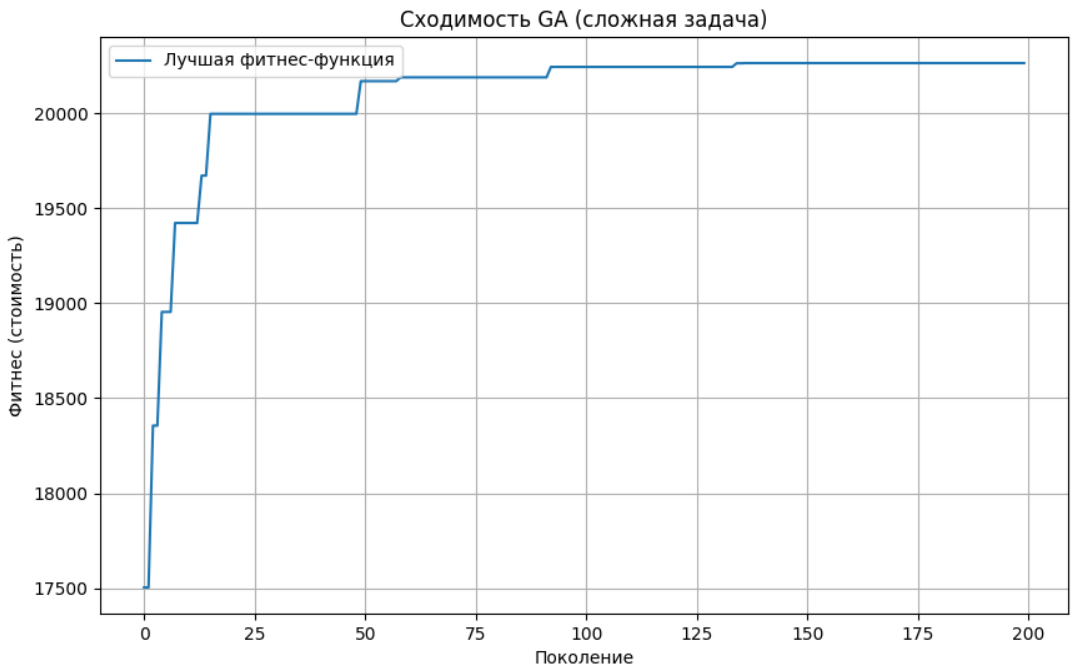
```
import numpy as np
import random
import matplotlib.pyplot as plt
import time

# Простая задача (набор 5)
CAPACITY_SIMPLE = 104
WEIGHTS_SIMPLE = [25, 35, 45, 5, 25, 3, 2, 2]
PROFITS_SIMPLE = [350, 400, 450, 20, 70, 8, 5, 5]
N_SIMPLE = len(WEIGHTS_SIMPLE)
OPTIMAL_SOLUTION_SIMPLE = [1, 0, 1, 1, 1, 0, 1, 1]
OPTIMAL_PROFIT_SIMPLE = 900

# Повышенная сложность (набор 2)
CAPACITY_COMPLEX = 13810
WEIGHTS_COMPLEX = [491, 925, 501, 832, 354, 883, 900, 462, 568, 921, 724, 487, 222,
325, 700, 167, 908, 269, 912, 310, 958, 707, 505, 518, 652, 588, 312, 208, 512, 935,
624, 76, 821, 726, 908, 192, 745, 59, 653, 274, 227, 876, 107, 523, 854, 245, 211,
881, 423, 717]
PROFITS_COMPLEX = [957, 948, 57, 85, 836, 736, 670, 309, 606, 607, 582, 159, 431, 394,
724, 995, 950, 545, 445, 269, 36, 28, 465, 319, 381, 892, 526, 561, 237, 24, 326, 137,
511, 999, 675, 182, 894, 797, 735, 907, 763, 790, 354, 981, 962, 162, 755, 716, 462,
531]
N_COMPLEX = len(WEIGHTS_COMPLEX)

class KnapsackGA:
```

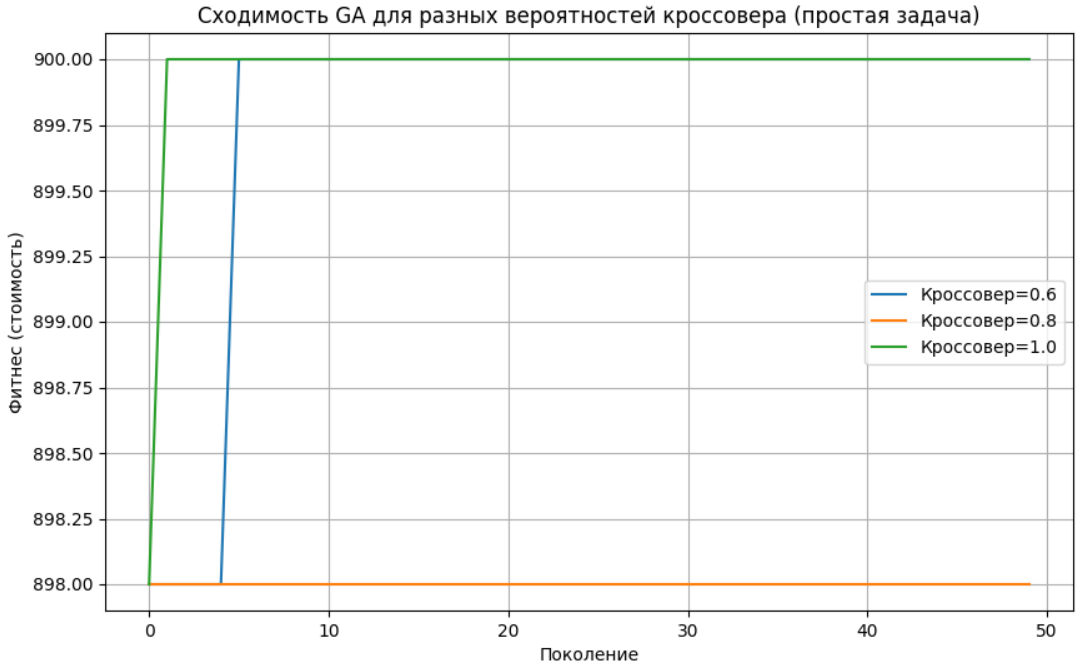
```python
    def __init__(self, capacity, weights, profits, pop_size=50, generations=100,
crossover_prob=0.8, mutation_prob=0.01, seed=42):
        np.random.seed(seed)
        random.seed(seed)
        self.capacity = capacity
        self.weights = weights
        self.profits = profits
        self.n = len(weights)
        self.pop_size = pop_size
        self.generations = generations
        self.crossover_prob = crossover_prob
        self.mutation_prob = mutation_prob
        self.population = self.initialize_population()
        self.fitness_history = []

    def initialize_population(self):
        return np.random.randint(0, 2, size=(self.pop_size, self.n))

    def fitness(self, individual):
        total_weight = np.sum(individual * self.weights)
        total_profit = np.sum(individual * self.profits)
        if total_weight > self.capacity:
            return total_profit - 1000 * (total_weight - self.capacity)
        return total_profit

    def tournament_selection(self, tournament_size=3):
        tournament = np.random.choice(self.pop_size, tournament_size)
        fitnesses = [self.fitness(self.population[i]) for i in tournament]
        return self.population[tournament[np.argmax(fitnesses)]].copy()

    def crossover(self, parent1, parent2):
        if np.random.rand() < self.crossover_prob:
            point = np.random.randint(1, self.n)
            child1 = np.concatenate((parent1[:point], parent2[point:]))
            child2 = np.concatenate((parent2[:point], parent1[point:]))
            return child1, child2
        return parent1.copy(), parent2.copy()

    def mutate(self, individual):
        for i in range(self.n):
            if np.random.rand() < self.mutation_prob:
                individual[i] = 1 - individual[i]
        return individual

    def run(self):
        for gen in range(self.generations):
            new_population = []
            best_idx = np.argmax([self.fitness(ind) for ind in self.population])
            new_population.append(self.population[best_idx].copy())

            while len(new_population) < self.pop_size:
                parent1 = self.tournament_selection()
                parent2 = self.tournament_selection()
```

```
                child1, child2 = self.crossover(parent1, parent2)
                new_population.append(self.mutate(child1))
                if len(new_population) < self.pop_size:
                    new_population.append(self.mutate(child2))

            self.population = np.array(new_population)
            best_fitness = max([self.fitness(ind) for ind in self.population])
            self.fitness_history.append(best_fitness)

        best_idx = np.argmax([self.fitness(ind) for ind in self.population])
        return self.population[best_idx], self.fitness_history

def plot_fitness(fitness_history, title, filename):
    plt.figure(figsize=(10, 6))
    plt.plot(fitness_history, label='Лучшая фитнес-функция')
    plt.title(title)
    plt.xlabel('Поколение')
    plt.ylabel('Фитнес (стоимость)')
    plt.grid(True)
    plt.legend()
    plt.savefig(filename)
    plt.close()

def run_simple_knapsack():
    ga = KnapsackGA(CAPACITY_SIMPLE, WEIGHTS_SIMPLE, PROFITS_SIMPLE, pop_size=50,
generations=100, crossover_prob=0.8, mutation_prob=0.01)
    best_solution, fitness_history = ga.run()
    total_weight = np.sum(best_solution * WEIGHTS_SIMPLE)
    total_profit = np.sum(best_solution * PROFITS_SIMPLE)
    print("Простая задача:")
    print(f"Лучшее решение: {best_solution}")
    print(f"Общий вес: {total_weight}")
    print(f"Общая стоимость: {total_profit}")
    print(f"Оптимальное решение: {OPTIMAL_SOLUTION_SIMPLE}")
    print(f"Оптимальная стоимость: {OPTIMAL_PROFIT_SIMPLE}")
    print(f"Точность: {total_profit / OPTIMAL_PROFIT_SIMPLE * 100:.2f}%")
    print(f"История фитнеса (выборка): {fitness_history[::10]}")
    plot_fitness(fitness_history, "Сходимость GA (простая задача)",
"simple_knapsack_convergence.png")
    return best_solution, fitness_history

def run_complex_knapsack():
    ga = KnapsackGA(CAPACITY_COMPLEX, WEIGHTS_COMPLEX, PROFITS_COMPLEX, pop_size=200,
generations=200, crossover_prob=0.8, mutation_prob=0.05)
    best_solution, fitness_history = ga.run()
    total_weight = np.sum(best_solution * WEIGHTS_COMPLEX)
    total_profit = np.sum(best_solution * PROFITS_COMPLEX)
    print("\nСложная задача:")
    print(f"Лучшее решение: {best_solution}")
    print(f"Общий вес: {total_weight}")
    print(f"Общая стоимость: {total_profit}")
    print(f"История фитнеса (выборка): {fitness_history[::40]}")
```

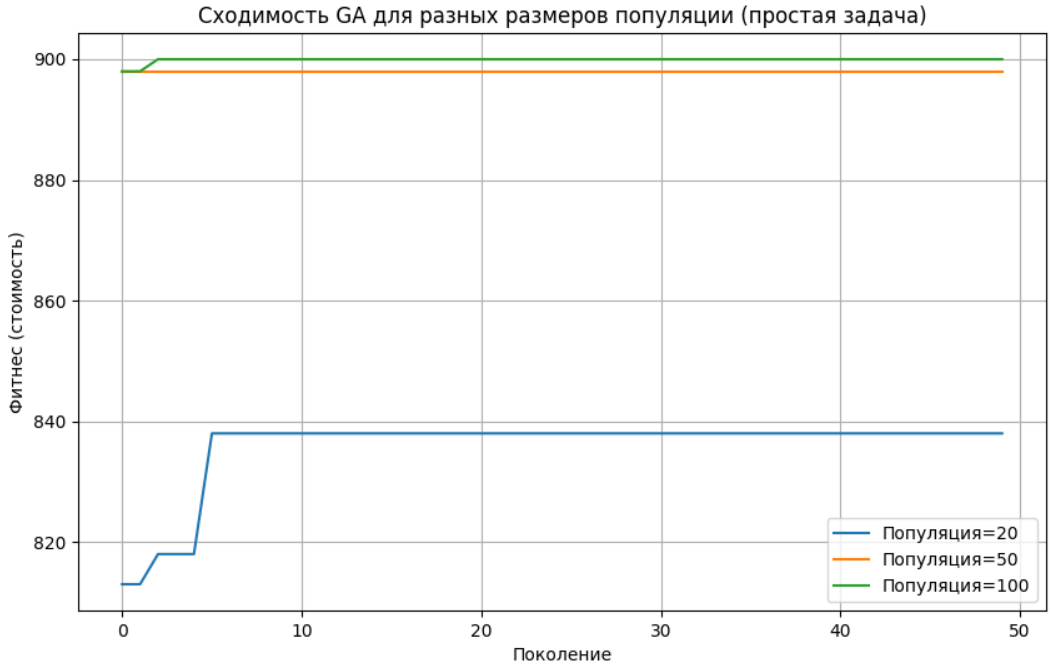
```
    plot_fitness(fitness_history, "Сходимость GA (сложная задача)",
"complex_knapsack_convergence.png")
    return best_solution, fitness_history

def analyze_parameters():
    pop_sizes = [20, 50, 100]
    crossover_probs = [0.6, 0.8, 1.0]
    mutation_probs = [0.001, 0.01, 0.1]

    print("\nАнализ параметров (простая задача):")

    # Анализ размера популяции
    fitness_histories_pop = []
    for pop_size in pop_sizes:
        start_time = time.time()
        ga = KnapsackGA(CAPACITY_SIMPLE, WEIGHTS_SIMPLE, PROFITS_SIMPLE,
pop_size=pop_size, generations=50)
        best_solution, fitness_history = ga.run()
        total_profit = np.sum(best_solution * PROFITS_SIMPLE)
        end_time = time.time()
        print(f"Популяция={pop_size}, Стоимость={total_profit}, Точность={total_profit
/ OPTIMAL_PROFIT_SIMPLE * 100:.2f}%, Время={end_time - start_time:.4f}s")
        fitness_histories_pop.append((pop_size, fitness_history))

    plt.figure(figsize=(10, 6))
    for pop_size, history in fitness_histories_pop:
        plt.plot(history, label=f'Популяция={pop_size}')
    plt.title('Сходимость GA для разных размеров популяции (простая задача)')
    plt.xlabel('Поколение')
    plt.ylabel('Фитнес (стоимость)')
    plt.grid(True)
    plt.legend()
    plt.savefig('population_convergence.png')
    plt.close()

    # Анализ вероятности кроссовера
    fitness_histories_cross = []
    for crossover_prob in crossover_probs:
        start_time = time.time()
        ga = KnapsackGA(CAPACITY_SIMPLE, WEIGHTS_SIMPLE, PROFITS_SIMPLE,
crossover_prob=crossover_prob, generations=50)
        best_solution, fitness_history = ga.run()
        total_profit = np.sum(best_solution * PROFITS_SIMPLE)
        end_time = time.time()
        print(f"Кроссовер={crossover_prob}, Стоимость={total_profit},
Точность={total_profit / OPTIMAL_PROFIT_SIMPLE * 100:.2f}%, Время={end_time -
start_time:.4f}s")
        fitness_histories_cross.append((crossover_prob, fitness_history))

    plt.figure(figsize=(10, 6))
    for crossover_prob, history in fitness_histories_cross:
        plt.plot(history, label=f'Кроссовер={crossover_prob}')
    plt.title('Сходимость GA для разных вероятностей кроссовера (простая задача)')
```

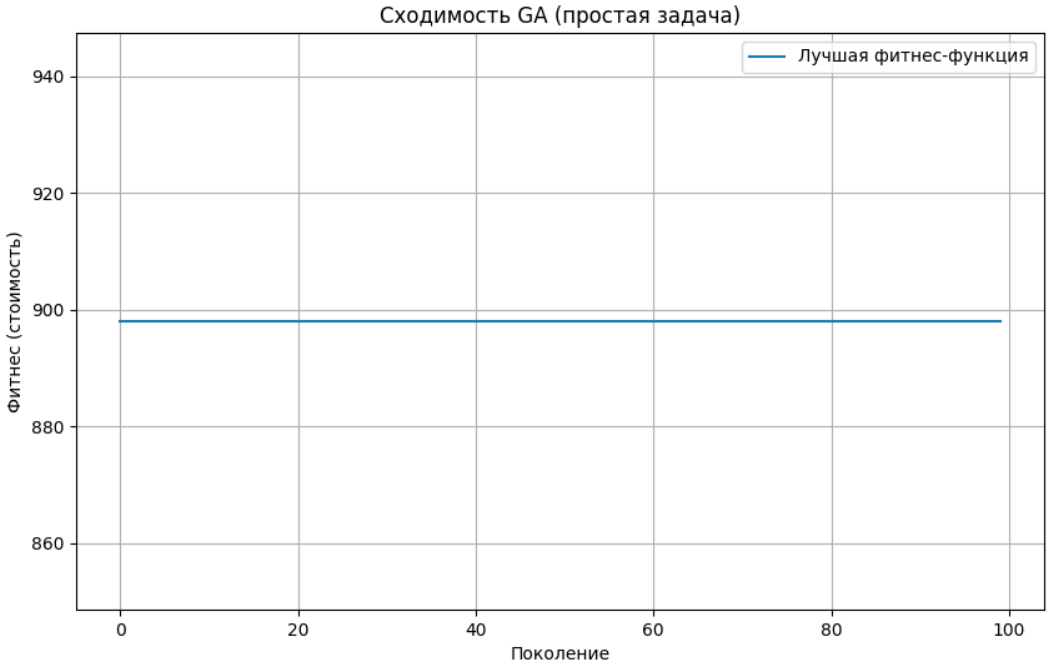
```
    plt.xlabel('Поколение')
    plt.ylabel('Фитнес (стоимость)')
    plt.grid(True)
    plt.legend()
    plt.savefig('crossover_convergence.png')
    plt.close()

    # Анализ вероятности мутации
    fitness_histories_mut = []
    for mutation_prob in mutation_probs:
        start_time = time.time()
        ga = KnapsackGA(CAPACITY_SIMPLE, WEIGHTS_SIMPLE, PROFITS_SIMPLE,
mutation_prob=mutation_prob, generations=50)
        best_solution, fitness_history = ga.run()
        total_profit = np.sum(best_solution * PROFITS_SIMPLE)
        end_time = time.time()
        print(f"Мутация={mutation_prob}, Стоимость={total_profit},
Точность={total_profit / OPTIMAL_PROFIT_SIMPLE * 100:.2f}%, Время={end_time -
start_time:.4f}s")
        fitness_histories_mut.append((mutation_prob, fitness_history))

    plt.figure(figsize=(10, 6))
    for mutation_prob, history in fitness_histories_mut:
        plt.plot(history, label=f'Мутация={mutation_prob}')
    plt.title('Сходимость GA для разных вероятностей мутации (простая задача)')
    plt.xlabel('Поколение')
    plt.ylabel('Фитнес (стоимость)')
    plt.grid(True)
    plt.legend()
    plt.savefig('mutation_convergence.png')
    plt.close()

if __name__ == "__main__":
    run_simple_knapsack()
    run_complex_knapsack()
    analyze_parameters()
```


Сходимость GA (сложная задача)
Лучшая фитнес-функция
Фитнес (стоимость)
20000
19500
19000
18500
18000
17500
0
25
50
75
100
125
150
175
200
Поколение


Сходимость GA для разных вероятностей кроссовера (простая задача)
Кроссовер=0.6
Кроссовер=0.8
Кроссовер=1.0
Фитнес (стоимость)
900.00
899.75
899.50
899.25
899.00
898.75
898.50
898.25
898.00
0
10
20
30
40
50
Поколение

Сходимость GA для разных вероятностей мутации (простая задача)
Фитнес (стоимость)
Поколение
Мутация=0.001
Мутация=0.01
Мутация=0.1
900.00
899.75
899.50
899.25
899.00
898.75
898.50
898.25
898.00
0
10
20
30
40
50


Сходимость GA для разных размеров популяции (простая задача)
Фитнес (стоимость)
Поколение
Популяция=20
Популяция=50
Популяция=100
900
880
860
840
820
0
10
20
30
40
50



Простая задача:

Лучшее решение: [1 0 1 1 1 1 0 0]

Общий вес: 103

Общая стоимость: 898

Оптимальное решение: [1, 0, 1, 1, 1, 0, 1, 1]

Оптимальная стоимость: 900

Точность: 99.78%

История фитнеса (выборка): [np.int64(898), np.int64(898), np.int64(898), np.int64(898), np.int64(898), np.int64(898), np.int64(898), np.int64(898), np.int64(898), np.int64(898), np.int64(898)]

Сложная задача:

Лучшее решение: [1 1 0 0 1 0 0 0 1 0 1 0 1 1 1 1 1 1 0 1 0 0 1 0 0 1 1 1 0 0 1 1 0 1 0 1 1

1 1 1 1 1 1 1 1 0 1 0 0 0]

Общий вес: 13723

Общая стоимость: 20263

История фитнеса (выборка): [np.int64(17505), np.int64(19996), np.int64(20188), np.int64(20243), np.int64(20263)]

Анализ параметров (простая задача):

Популяция=20, Стоимость=838, Точность=93.11%, Время=0.0265s

Популяция=50, Стоимость=898, Точность=99.78%, Время=0.0650s

Популяция=100, Стоимость=900, Точность=100.00%, Время=0.1286s

Кроссовер=0.6, Стоимость=900, Точность=100.00%, Время=0.0642s

Кроссовер=0.8, Стоимость=898, Точность=99.78%, Время=0.0657s

Кроссовер=1.0, Стоимость=900, Точность=100.00%, Время=0.0724s

Мутация=0.001, Стоимость=898, Точность=99.78%, Время=0.0644s

Мутация=0.01, Стоимость=898, Точность=99.78%, Время=0.0646s

Мутация=0.1, Стоимость=900, Точность=100.00%, Время=0.0652s

Ответ на контрольный вопрос:

Опишите кодирование переменной длины для задачи укладки рюкзака.

Кодирование переменной длины для задачи укладки рюкзака предполагает использование хромосом, длина которых может изменяться в зависимости от количества предметов, включенных в решение. В отличие от фиксированной длины, где каждому предмету соответствует фиксированный бит (0 или 1), в этом подходе хромосома представляет собой список индексов или идентификаторов тех предметов, которые выбраны для укладки в рюкзак. Длина хромосомы варьируется, так как она зависит от числа выбранных предметов, а не от общего числа доступных предметов. Этот метод позволяет гибко адаптироваться к ограничениям по весу, добавляя или удаляя предметы в процессе эволюции, но требует специальных операторов скрещивания и мутации, чтобы сохранять допустимость решений и управлять длиной хромосомы.

Вывод:

В ходе выполнения лабораторной работы я научился применять генетические алгоритмы для решения задачи укладки рюкзака как простой, так и повышенной сложности, что позволило мне глубже понять принципы эволюционных вычислений. Я освоил настройку параметров алгоритма, таких как размер популяции, вероятности кроссовера и мутации, и увидел, как они влияют на точность и скорость сходимости решения. Также я научился визуализировать результаты с помощью графиков сходимости и анализировать время выполнения, что помогло мне оценить эффективность различных конфигураций ГА. Эта работа укрепила мои навыки программирования и дала практический опыт работы с оптимизационными задачами.